

Table of Contents

1. Executive Summary	3
2. Architecture Overview	3
2.1 Three-Layer Architecture	3
2.2 Communication Flow	4
3. Integrated Companies and APIs	4
3.1 LLM Providers (AI Brain)	4
3.2 TTS (Text-to-Speech) Providers	5
3.3 STT (Speech-to-Text) Providers	5
3.4 Other Integrated Services	5
4. Current Features (What Exists)	6
4.1 AI Chat System	6
4.2 Task Classification and Agent Routing	6
4.3 Sub-Agent System (Cloud)	6
4.4 Desktop Automation (Electron)	7
4.5 Desktop Automation (Python)	7
4.6 Voice System	7
4.7 Multi-AI Research	8
4.8 Auto-Update System	8

4.9 Electron UI Pages	8
5. What Is Missing / Needs Work	9
5.1 Critical: Voice/Mic Not Working in Electron	9
5.2 Critical: Automation Not Executing from Chat	9
5.3 Critical: NSIS Installer Not Building	9
5.4 Major: Python Agent Not Connected to Electron UI	10
5.5 Major: Browser Agent Has No Real Browser	10
5.6 Major: Memory Is In-Memory Only	10
5.7 Minor Issues	10
6. Complete File Inventory	11
6.1 Cloud Backend (Next.js)	11
6.2 Desktop Application (Electron)	11
6.3 Python Desktop Agent	12
7. Recommended Fixes (Priority Order)	13

1. Executive Summary

JARVIS-HYBRID is a comprehensive AI-powered desktop assistant built with a hybrid cloud-desktop architecture. The project combines a Next.js 16 cloud backend with an Electron desktop application and a Python desktop agent to deliver real-time system automation, voice interaction, freelance job hunting, WhatsApp communication, and multi-provider AI chat capabilities. The system is designed to act as "Ryan" (a real human freelancer) when communicating with clients, never revealing its AI identity. The current version is 2.0.0, deployed on Vercel for the cloud component and packaged as a Windows desktop application via Electron Builder.

The project repository is hosted at github.com/mohsinrasheed-Baga1/jarvis-hydird-update and is connected to GitHub for version control and auto-update distribution. The architecture follows a "Cloud Brain + Desktop Hands" pattern where the cloud handles AI reasoning, LLM routing, and task classification, while the desktop components handle system-level operations like opening applications, sending WhatsApp messages, taking screenshots, and controlling system volume. This separation allows the system to work both online and offline, with the desktop agent capable of operating independently when the cloud is unreachable.

2. Architecture Overview

2.1 Three-Layer Architecture

JARVIS-HYBRID employs a three-layer architecture that separates concerns between cloud intelligence, desktop automation, and user interface presentation. The Cloud Layer (Next.js 16 on Vercel) provides the AI brain with LLM routing, task classification, and agent orchestration. The Desktop Layer (Electron + Python) provides system-level automation capabilities including file operations, window management, WhatsApp control, and voice input/output. The UI Layer (React + Vite within Electron) provides the user-facing interface with chat, automation panels, voice controls, and settings management.

Layer	Technology	Role	Key Files
Cloud Backend	Next.js 16, Vercel	AI Brain, LLM Routing, Agent Core	app/api/*, lib/*
Desktop Shell	Electron 34, electron-builder	System Access, Auto-Update, IPC	desktop-app/electron/main.js
Desktop UI	React 19, Vite, Tailwind	User Interface, Chat, Settings	desktop-app/src/*

Python Agent	Python 3, pyautogui	Desktop Automation, WhatsApp, Jobs	desktop/jarvis/*
Cloud-Python Bridge	HTTP polling, REST API	Cloud-to-Desktop task routing	desktop/jarvis/connector.py

Table 1: Architecture Layers and Their Roles

2.2 Communication Flow

The system uses a polling-based communication model between the cloud and desktop components. When a user sends a message through the Electron UI, it hits the Cloud Backend API (/api/chat). The Agent Core classifies the task and routes it to the appropriate sub-agent. If the task requires local execution (e.g., opening an app, taking a screenshot, sending a WhatsApp message), it is queued in the /api/agent endpoint. The Python Desktop Connector polls this endpoint every 2 seconds, picks up pending tasks, executes them locally, and reports results back via PUT /api/agent. For direct desktop actions within Electron, the renderer communicates with the main process via IPC (contextBridge + ipcRenderer), which then uses shell.openExternal, child_process, or PowerShell commands to perform system operations.

3. Integrated Companies and APIs

3.1 LLM Providers (AI Brain)

JARVIS-HYBRID integrates with six major LLM providers through its LLMRouter system. The router implements automatic failover: if the primary provider fails or hits a rate limit, it seamlessly switches to the next available provider. Multi-key support allows comma-separated API keys per provider, with automatic rotation on 429 (rate limit) responses. This ensures near-zero downtime for AI conversations. The classification system (classifyTask) uses a lightweight LLM call with temperature 0.1 to determine which agent should handle the user request before processing.

Provider	Primary Model	Fallback Model	Capability	API Key Prefix
Groq	llama-3.3-70b-versatile	llama-3.1-8b-instant	Chat + Streaming + STT	gsk_...
Google Gemini	gemini-1.5-flash	gemini-1.5-pro	Chat + Research	AIza...
OpenAI	gpt-4o-mini	gpt-3.5-turbo	Chat + Streaming + STT + TTS	sk-...

ZAI (GLM)	glm-4-flash	glm-4-flash	Chat (OpenAI-compatible)	custom
xAI / Grok	grok-2	grok-2-mini	Chat + Research	xai-...
Anthropic	claude-3-5-sonnet	claude-3-haiku	Chat + Research	sk-ant-...

Table 2: LLM Provider Integration Details

3.2 TTS (Text-to-Speech) Providers

The TTS system follows a priority chain with automatic fallback: ElevenLabs Turbo v2.5 (best for Hindi/Urdu, supports emotion-based voice settings) is tried first, then Sarvam AI (Indian TTS with natural Hindi/Urdu), then OpenAI TTS HD (high-quality English), then Google Translate TTS (free but robotic), and finally StreamElements as the last resort. Each provider supports multi-key rotation with automatic failover on rate limits. The ElevenLabs integration includes emotion-aware voice settings with configurable stability, similarity, and style parameters for each emotion type (happy, serious, sympathetic, surprised, encouraging, normal).

Provider	Model	Priority	Language Support	Quality
ElevenLabs	eleven_turbo_v2.5	1 (Best)	Hindi/Urdu + English	Natural, Human-like
Sarvam AI	bulbul:v1	2	Hindi/Urdu (hi-IN)	Natural Indian TTS
OpenAI	tts-1-hd	3	Multi-language	High Quality
Google Translate	TTS API	4 (Free)	Multi-language	Robotic
StreamElements	Kappa v2	5 (Last)	Limited	Basic

Table 3: TTS Provider Priority Chain

3.3 STT (Speech-to-Text) Providers

The voice recognition system operates in two modes depending on the runtime environment. In the browser (web), it uses the native Web Speech API (webkitSpeechRecognition). In the Electron desktop app, where Web Speech API does not work, it falls back to cloud-based Whisper models via the /api/voice endpoint. Groq Whisper (whisper-large-v3-turbo) is tried first for its speed, with OpenAI Whisper (whisper-1) as backup. The Electron main process also includes a Windows Speech API listener via PowerShell (System.Speech.Recognition) for offline voice recognition on Windows, accessible through the listen-once IPC handler.

3.4 Other Integrated Services

Service	Purpose	Integration Point
GitHub	Auto-update distribution, version control	electron-updater + Git polling
GitHub API	Update checking, commit tracking	AutoUpdater._check_github_api()
WhatsApp Desktop	Messaging automation, client communication	Python pyautogui + pyperclip
Vercel	Cloud hosting for Next.js backend	Automatic deployment from GitHub
Freelance Platforms	Job hunting (Upwork, Fiverr, LinkedIn)	LLM-based search + proposals
PyAutoGUI	Desktop GUI automation	Screenshots, mouse/keyboard control
PowerShell	Windows system commands	Volume, screenshots, notifications

Table 4: Additional Service Integrations

4. Current Features (What Exists)

4.1 AI Chat System

The core chat system is fully functional with streaming support. Users can converse with JARVIS in Urdu, English, or mixed languages, and the AI responds in the same language with natural, human-like personality. The system prompt enforces strict rules: JARVIS speaks as a real person (not as AI), uses filler words and emotions naturally, and never writes robotic phrases like "How can I assist you?" The chat supports file uploads via multipart form data, with images and documents analyzed by the AI. Conversation history is maintained in-memory (MemoryManager) with per-user isolation and up to 100 messages per user.

4.2 Task Classification and Agent Routing

The LLMRouter.classifyTask() method uses a lightweight LLM call to classify user messages into one of 9 agent types: general, windows, browser, file, product_hunter, code, upload, freelance, whatsapp, and task_manager. Each classification includes the agent name, action, parameters, and confidence score. Urdu keywords are mapped to appropriate agents. The classification result determines which sub-agent handles the request, enabling specialized processing for different types of tasks.

4.3 Sub-Agent System (Cloud)

Agent	Actions	Status
-------	---------	--------

BrowserAgent	search, scrape, summarize	LLM-based, no real browser
CodeAgent	write, debug, review	Implemented (LLM-generated)
FreelanceAgent	hunt_jobs, apply, proposal, negotiate, cover_letter, portfolio_pitch, full_pipeline	Fully implemented (12 actions)
WhatsAppAgent	draft_message, auto_reply, client_chat, strategy, professional_reply, friendly_reply, negotiate, follow_up	Fully implemented (text only)
ProductHunterAgent	Product research, trending analysis	Implemented (LLM-based)
TaskManager	plan, execute_freelance, execute_whatsapp, quick_proposal, daily_plan	Fully implemented

Table 5: Cloud Sub-Agents and Their Capabilities

4.4 Desktop Automation (Electron)

The Electron main process (main.js) provides comprehensive desktop automation through IPC handlers and the `executeDesktopAction()` function. Supported actions include: opening URLs in the default browser, Google search, YouTube search and playback, opening applications (with name normalization for common apps like Chrome, Notepad, Calculator), volume control (up, down, mute) via PowerShell `SendKeys`, screenshot capture via PowerShell, system commands (lock screen), opening folders, and desktop notifications. The `preload.js` exposes all these as typed methods on `window.electronAPI` for the React renderer to call.

4.5 Desktop Automation (Python)

The Python desktop agent provides deeper system integration through the `LocalWindowsAgent`, `WhatsAppAgent`, `JobSearchAgent`, and `AutoUpdater`. The Windows agent handles: screenshots, app launching, URL opening, Google/YouTube search, text typing, hotkey pressing, mouse clicking, system info, volume/brightness control, screen lock, shutdown/restart, clipboard read/write, and desktop notifications. The WhatsApp agent uses `pyautogui` and `pypierclip` to automate the WhatsApp Desktop application: opening it, searching contacts, sending messages, reading chats via OCR, and generating human-like replies as "Ryan". The `AutoUpdater` checks GitHub for new commits and can pull updates automatically.

4.6 Voice System

The voice system supports both input (STT) and output (TTS). For input: the `VoiceService` in the Electron app uses `MediaRecorder` to capture audio, then sends it to `/api/voice` for cloud Whisper

transcription. The Electron main process also has a Windows Speech API listener via PowerShell for offline recognition. For output: the TTS API follows the priority chain described in Section 3.2, with the `VoiceService.speak()` method trying natural voices first and falling back to browser `SpeechSynthesisUtterance`. The Python agent has its own `VoiceEngine` and `STTEngine` for local voice interaction.

4.7 Multi-AI Research

The `/api/research` endpoint enables consulting multiple AI providers simultaneously. It sends the same query to all configured providers (Groq, Gemini, OpenAI, xAI, Anthropic) in parallel using `Promise.allSettled()`, then returns combined results with a summary. This allows users to compare answers from different AI models for research and analysis tasks.

4.8 Auto-Update System

Two separate auto-update mechanisms exist: (1) Electron auto-updater using `electron-updater`, which checks GitHub Releases for new versions, downloads NSIS/portable updates, and prompts for installation on app restart; (2) Python `AutoUpdater` using `git fetch/pull`, which checks for new commits, pulls updates, installs dependencies, and can restart the Python agent automatically. Both systems support periodic checking (every 30 minutes by default) and manual triggers.

4.9 Electron UI Pages

Page	File	Purpose
Chat	ChatPage.tsx	Main chat interface with streaming responses
Automation	AutomationPage.tsx	Task monitoring and quick actions panel
Voice	VoicePage.tsx	Voice input/output configuration
Files	FilesPage.tsx	File browser and operations
Memory	MemoryPage.tsx	Conversation history and preferences
Settings	SettingsPage.tsx	API key management, provider selection
Status	StatusPage.tsx	System status dashboard

Table 6: Electron UI Pages

5. What Is Missing / Needs Work

5.1 Critical: Voice/Mic Not Working in Electron

The most critical issue is that the voice/microphone system does not work properly in the Electron environment. `webkitSpeechRecognition` is not available in Electron, and the cloud Whisper fallback path may not trigger correctly. The `VoiceService` uses `MediaRecorder` + cloud STT, but there are potential issues with: (a) microphone permissions not being properly granted despite the `session.setPermissionRequestHandler` in `main.js`, (b) audio encoding issues when sending recorded audio to the Whisper API, (c) the UI not clearly indicating which STT method is active. The Electron `main.js` includes a Windows Speech API listener via PowerShell, but this is only accessible through the listen-once IPC handler and may not be connected to the voice UI.

5.2 Critical: Automation Not Executing from Chat

The system prompt tells the AI it can execute desktop actions ("YouTube kholna", "volume up", "screenshot"), and the Electron `main.js` has the `executeDesktopAction()` function with full IPC support. However, the critical missing piece is the action extraction and execution pipeline from AI chat responses. When a user says "open YouTube and play tilawat", the AI generates a text response like "I will open YouTube for you" but does NOT output a structured action command that the frontend can parse and send to the desktop-action IPC handler. The system prompt mentions desktop actions but does not instruct the AI to output them in a machine-readable format (e.g., JSON with type, query, etc.). The frontend chat component receives the text response and displays it, but never scans for action commands to execute via `window.electronAPI.desktopAction()`.

Additionally, the `AutomationPage.tsx` only has hardcoded demo data and quick action buttons that are not connected to actual desktop actions. The `taskService.ts` simulates task execution with `setTimeout` and does not delegate to the real `electronAPI` methods.

5.3 Critical: NSIS Installer Not Building

The `package.json` configures both NSIS and portable targets for Windows builds, and `electron-updater` requires NSIS for auto-update support. However, building the NSIS installer on Linux (the deployment environment) fails because it requires Wine. The portable build works but does not support auto-update. The `nsis` configuration in `package.json` is comprehensive (`oneClick`, `allowToChangeInstallationDirectory`, `installerLanguages` including `ur_PK`), but it cannot be generated without a Windows build environment. This means auto-update from GitHub Releases is currently non-functional for end users.

5.4 Major: Python Agent Not Connected to Electron UI

The Python desktop agent (desktop/jarvis/) has its own CLI and GUI (CustomTkinter) interfaces, but it is not properly connected to the Electron UI. The Electron main.js attempts to launch the Python agent as a subprocess, but the Python agent runs independently with its own connector that polls the cloud API. There is no direct IPC bridge between the Electron renderer and the Python agent. This means features like WhatsApp automation, job search, and auto-update status from the Python agent are not reflected in the Electron UI. The user must interact with the Python agent separately through its own interface.

5.5 Major: Browser Agent Has No Real Browser

The BrowserAgent in the cloud backend does not actually browse the web. Its search, scrape, and summarize actions all use LLM-generated responses based on the model training data, not real-time web access. The search action generates a response "as if it had access to the latest information," the scrape action "provides what you know about this URL," and the summarize action similarly relies on training data. There is no Playwright, Puppeteer, or any headless browser integration for actual web scraping or real-time search. This severely limits the freelance job hunting capability, as the agent cannot actually visit Upwork, Fiverr, or LinkedIn to find real jobs.

5.6 Major: Memory Is In-Memory Only

The MemoryManager uses in-memory Maps (conversations, userPreferences, errorPatterns) that reset on every cold start. On Vercel, this means conversation history is lost whenever the serverless function spins down. The code comments mention "use Vercel KV for production" but no persistent storage is implemented. This affects user experience as conversation context is not preserved between sessions.

5.7 Minor Issues

Issue	Impact	Fix Complexity
Upload agent requires browser automation	Upload to Redbubble/Amazon/Etsy not possible	High (needs Playwright)
WhatsApp uses hardcoded coordinates	Only works on specific screen resolutions	Medium (need dynamic detection)
No WebSocket for real-time sync	2-second polling delay for task routing	Medium
No multi-user authentication	No user auth system	Medium

Desktop connector sends empty API keys	Cloud chat from Python agent lacks API keys	Low
No error recovery for failed subprocesses	Cloud/Python failures cause silent degradation	Medium

Table 7: Minor Issues and Their Impact

6. Complete File Inventory

6.1 Cloud Backend (Next.js)

File	Purpose
app/api/chat/route.ts	Main chat API with streaming and file upload
app/api/agent/route.ts	Cloud-to-desktop task queue (GET/POST/PUT)
app/api/voice/route.ts	STT via Groq Whisper + OpenAI Whisper
app/api/tts/route.ts	TTS priority chain (ElevenLabs > Sarvam > OpenAI > Google)
app/api/research/route.ts	Multi-AI research consultation (5 providers)
app/api/health/route.ts	Health check endpoint
app/api/memory/route.ts	Conversation history and preferences CRUD
lib/agent-core.ts	Agent orchestrator with emotion detection and sub-agent routing
lib/llm-router.ts	Multi-provider LLM router with failover and multi-key
lib/protocol.ts	Shared type definitions (AgentType, APIKeys, LLMConfig)
lib/memory.ts	In-memory conversation and preference storage
lib/sub-agents/browser-agent.ts	Web search/scraping agent (LLM-based)
lib/sub-agents/freelance-agent.ts	Freelance job hunting with 12 actions
lib/sub-agents/whatsapp-agent.ts	WhatsApp message drafting and strategy agent
lib/sub-agents/code-agent.ts	Code writing/debugging agent
lib/sub-agents/task-manager.ts	Autonomous task planner and executor

Table 8: Cloud Backend Files

6.2 Desktop Application (Electron)

File	Purpose
desktop-app/electron/main.js	Electron main process with desktop automation IPC
desktop-app/electron/preload.js	Context bridge exposing electronAPI to renderer
desktop-app/src/App.tsx	Main app with sidebar navigation and update bar
desktop-app/src/pages/ChatPage.tsx	Chat interface with streaming support
desktop-app/src/pages/AutomationPage.tsx	Task monitoring panel (demo data only)
desktop-app/src/pages/VoicePage.tsx	Voice configuration page
desktop-app/src/pages/SettingsPage.tsx	API key and provider settings
desktop-app/src/services/apiClient.ts	API client for cloud backend
desktop-app/src/services/voiceService.ts	Voice recording, transcription, TTS playback
desktop-app/src/services/taskService.ts	Task polling and execution (simulated)
desktop-app/package.json	Electron app config (v2.0.0) with NSIS build targets

Table 9: Electron Desktop App Files

6.3 Python Desktop Agent

File	Purpose
desktop/jarvis/main.py	Main entry: CLI, GUI, and background modes
desktop/jarvis/connector.py	Cloud connector with HTTP polling task execution
desktop/jarvis/config.json	Configuration: API keys and skills
desktop/jarvis/local_agents/windows_agent.py	System control: apps, screenshots, volume, keyboard
desktop/jarvis/local_agents/whatsapp_agent.py	WhatsApp Desktop automation via pyautogui
desktop/jarvis/local_agents/job_search_agent.py	Multi-platform job search agent
desktop/jarvis/local_agents/file_agent.py	File operations (read, write, download, delete)
desktop/jarvis/local_agents/auto_update.py	Git-based auto-update with rollback support
desktop/jarvis/voice/voice_engine.py	Local TTS/STT engine

Table 10: Python Desktop Agent Files

7. Recommended Fixes (Priority Order)

Priority	Fix	Description	Effort
P0	Action Extraction Pipeline	Update system prompt to output JSON action commands. Add frontend parser to detect and execute via electronAPI.	3-5 days
P0	Fix Voice in Electron	Ensure cloud Whisper triggers when webkitSpeechRecognition unavailable. Fix mic permissions.	2-3 days
P0	NSIS Installer on Windows	Set up GitHub Actions CI/CD on Windows runner to build NSIS installer for auto-update.	1-2 days
P1	Connect Python to Electron UI	Add WebSocket bridge between Electron main process and Python agent for real-time status.	3-5 days
P1	Real Browser for Job Search	Integrate Playwright headless browser for actual web scraping of Upwork/Fiverr.	5-7 days
P1	Persistent Memory	Replace in-memory Maps with Vercel KV, SQLite, or local JSON persistence.	2-3 days
P2	Dynamic WhatsApp Automation	Replace hardcoded coordinates with accessibility-based element detection.	3-5 days
P2	WebSocket for Cloud-Desktop	Replace HTTP polling with WebSocket for real-time task routing.	2-3 days

Table 11: Recommended Fixes with Priority and Effort Estimates

The most impactful fix is the Action Extraction Pipeline (P0). This is the missing link between the AI brain and the desktop hands. Currently, the AI generates human-readable text responses but never outputs structured commands that the frontend can execute. The fix requires: (1) updating the system prompt to instruct the AI to append a JSON action block when it detects a desktop command, (2) adding a response parser in the ChatPage component that scans for action blocks and calls `window.electronAPI.desktopAction()`, and (3) showing execution results back in the chat. This single fix would make the existing desktop automation (YouTube, Google, apps, volume, screenshots) actually functional from the chat interface.